

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN AREQUIPA
FACULTAD DE INGENIERÍA DE PRODUCCIÓN DE
SERVICIOS
ESCUELA PROFESIONAL DE CIENCIA DE LA
COMPUTACIÓN



**Programa Interactivo del Algoritmo Genético
para el TSP**

Curso:

Tópicos en Inteligencia Artificial

Alumnos:

Ferro Vásquez, Arleen Maritza
Huanca Olazabal, Cristhian David
Mamani Casilla, Wilson Isaac

Docente:

Juan Carlos Gutierrez Cáceres

Semestre:

IX

AREQUIPA - PERÚ
2026

1. Introducción

El presente informe documenta el desarrollo y uso de un simulador interactivo para la resolución del **Problema del Viajante de Comercio** (TSP, por sus siglas en inglés) mediante el uso de un **Algoritmo Genético**.

Esta herramienta ha sido desarrollada en C++ utilizando OpenGL para el renderizado gráfico y la biblioteca Dear ImGui para la interfaz de usuario. El objetivo principal de la aplicación es proporcionar un entorno educativo y de laboratorio donde el usuario pueda modificar en tiempo real los parámetros y operadores del algoritmo genético para observar cómo estas variables afectan la convergencia y el rendimiento en la búsqueda de la ruta óptima.

2. Arquitectura y Tecnologías

La herramienta se compone de varias partes interconectadas:

- **Motor Genético:** Encargado de la lógica matemática. Implementa la representación de individuos (rutas), cálculo de aptitud (distancia total), métodos de inicialización (Aleatorio vs. Greedy), selección (Torneo, Ruleta), cruce (OX, PMX) y mutación (Swap, Inversión).
- **Renderizado Visual:** Utiliza *OpenGL* a través de *GLFW* para dibujar el grafo completo de las ciudades, la mejor ruta encontrada en la generación actual y datos de estado en pantalla.
- **Panel de Control:** Implementado con *Dear ImGui*, proporciona controles deslizantes, botones y menús desplegables para alterar la configuración de forma dinámica sin necesidad de recompilar.

3. Manual de Uso y Configuración del Algoritmo

La aplicación ofrece un entorno interactivo dividido en dos áreas principales: el lienzo de simulación a la izquierda, donde se visualizan las ciudades y las rutas, y el panel de configuración a la derecha, donde el usuario interactúa con los hiperparámetros del algoritmo.

3.1. Controles de Simulación

Para facilitar el estudio del algoritmo, se han implementado controles directos sobre la ejecución:

- **Reanudar / Pausar:** Permite detener la evolución en una generación específica para analizar la ruta actual detalladamente.
- **Paso a Paso:** Avanza exactamente una generación a la vez, ideal para observar pequeños cambios causados por la mutación o el cruce.
- **Reiniciar Simulación:** Genera un nuevo entorno geográfico (nuevas coordenadas para las ciudades) y reinicia el contador de generaciones y la población.

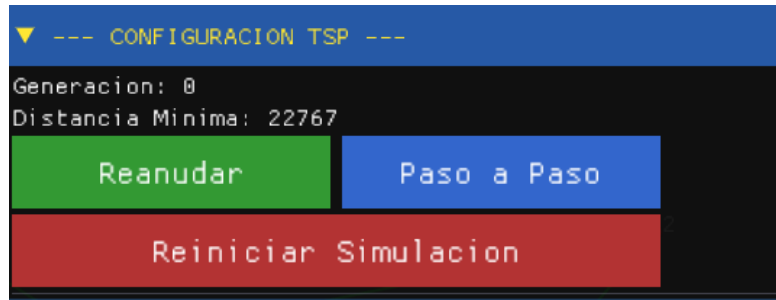


Figura 1: Panel de controles de simulación.

3.2. Estructura e Inicialización

El primer bloque de configuración permite definir el tamaño del problema y cómo se generará la primera generación de soluciones:

- **Número de Ciudades y Población:** A mayor número de ciudades, el espacio de búsqueda crece factorialmente. Un tamaño de población más grande ayuda a mantener la diversidad genética, evitando caer en mínimos locales tempranos, pero incrementa el costo computacional.
- **Modo de Inicialización:**
 - *Aleatoria:* Crea rutas al azar. Fomenta una exploración amplia desde el principio, pero comienza con distancias muy subóptimas.
 - *Greedy (Vecino Más Cercano):* Genera individuos conectando siempre la ciudad actual con la más cercana. Esto inyecta conocimiento heurístico en la población inicial, logrando una convergencia mucho más rápida, aunque aumenta el riesgo de estancamiento temprano.

3.3. Operadores Genéticos

El corazón del simulador radica en poder intercambiar los operadores biológicos simulados en tiempo real:

3.3.1. Selección

Define cómo se eligen los padres para la siguiente generación:

- **Torneo:** Se seleccionan K individuos al azar y el mejor de ellos gana el derecho a reproducirse. Un torneo más grande (mayor K) aumenta la presión selectiva, favoreciendo a los mejores individuos rápidamente a costa de perder diversidad.
- **Ruleta:** La probabilidad de selección es proporcional al *fitness* (inverso de la distancia). Es más justa con los individuos promedio, manteniendo mayor diversidad genética.

3.3.2. Cruce (Recombinación)

Combina el material genético de dos padres para crear descendencia, evitando ciudades duplicadas o faltantes (condición esencial en el TSP):

- **OX (Order Crossover):** Extrae una subsecuencia de un padre y rellena el resto con las ciudades del otro padre manteniendo su orden relativo. Excelente para preservar el orden general de las rutas.
- **PMX (Partially Mapped Crossover):** Mapea relaciones entre segmentos de dos padres. Suele ser muy efectivo para el TSP ya que preserva tanto la posición absoluta como el orden relativo de muchas ciudades.

3.3.3. Mutación

Introduce variabilidad aleatoria para evitar que toda la población converja prematuramente a un mismo camino ineficiente:

- **Swap (Intercambio):** Intercambia de posición dos ciudades aleatorias en la ruta. Es una mutación suave.
- **Inversión (2-opt):** Selecciona un segmento de la ruta y lo invierte. Es altamente destructivo pero increíblemente eficiente en el TSP para desenredar cruces de caminos en el grafo visual.
- **Tasa de Mutación:** Un valor muy bajo hace que el algoritmo se estanque. Un valor muy alto lo convierte en una búsqueda aleatoria sin rumbo. El valor óptimo suele estar entre 0,01 y 0,05.

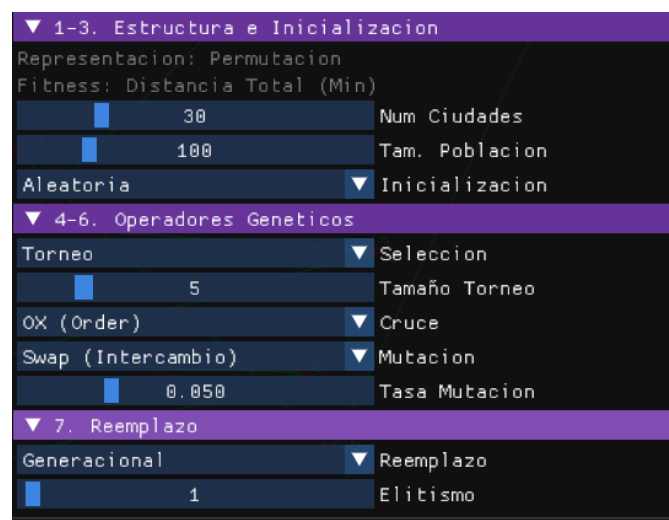


Figura 2: Configuración en tiempo real de los Operadores Genéticos.

3.4. Estrategias de Reemplazo y Elitismo

Finalmente, el simulador permite decidir cómo se forma la nueva generación:

- **Modelo Generacional vs Estacionario:** El modelo generacional reemplaza a toda la población a la vez, mientras que el estacionario solo reemplaza a un pequeño porcentaje de los peores individuos (ej. el 10 %) con nuevos hijos, lo que suele ser más conservador y rápido iterativamente.
- **Elitismo:** Garantiza que los mejores individuos de la generación actual pasen intactos a la siguiente. Sin elitismo, el algoritmo podría "olvidar" la mejor solución encontrada debido a una mutación destructiva accidental.

3.5. Evolución Visual y Búsqueda de Rutas

El mayor valor educativo del simulador gráfico es la capacidad de observar en tiempo real cómo la malla de la ruta evoluciona y se "desenreda" generación tras generación.

Caso de Prueba y Resultados

Para demostrar el funcionamiento de la herramienta, se configuró un escenario empírico con los siguientes parámetros:

- **Número de Ciudades:** 50
- **Población:** 100 individuos
- **Inicialización:** Aleatoria
- **Selección:** Torneo
- **Cruce:** OX (Order Crossover)
- **Mutación:** Inversión (2-opt) con una tasa del 0.05 (5 %)
- **Reemplazo:** Generacional con elitismo de 1

A lo largo de la simulación, se registraron cinco momentos clave de la evolución, demostrando una drástica caída inicial en la distancia total, seguida de un proceso de refinamiento fino (convergencia asintótica). Los resultados se resumen en la Tabla 1.

Momento	Generación	Distancia Mínima
1	0	26463
2	15	17622
3	70	9824
4	150	7850
5	1050	6962

Cuadro 1: Evolución de la distancia mínima a través de las generaciones.

A continuación, se presentan los espacios para las capturas gráficas correspondientes a estos cinco momentos, donde se puede apreciar visualmente cómo el algoritmo y la mutación 2-opt eliminan progresivamente los cruces de las rutas:

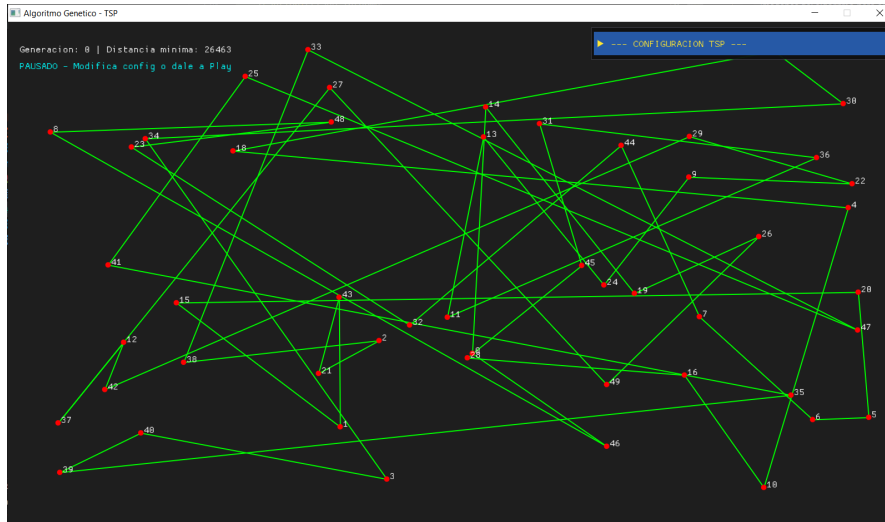


Figura 3: Generación 0: Rutas completamente aleatorias con un alto grado de intersecciones (Distancia: 26463).

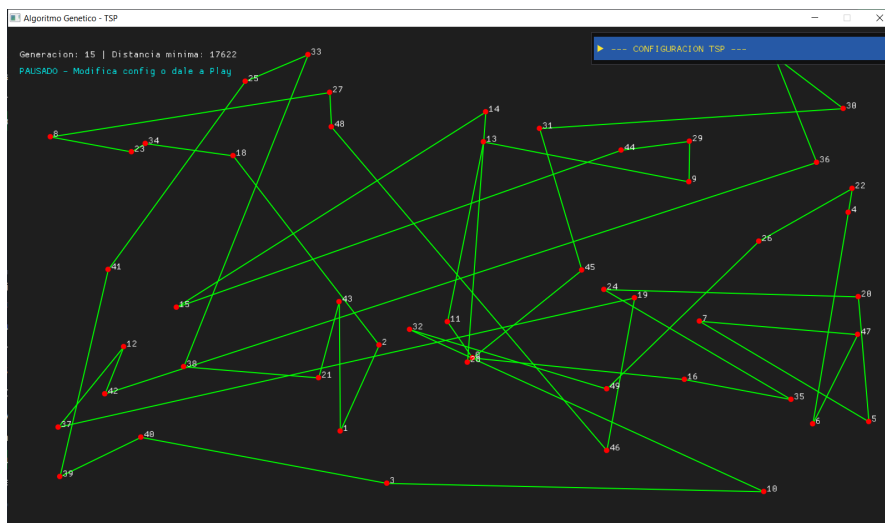


Figura 4: Generación 15: Rápida mejora inicial desenredando los cruces locales más obvios (Distancia: 17622).

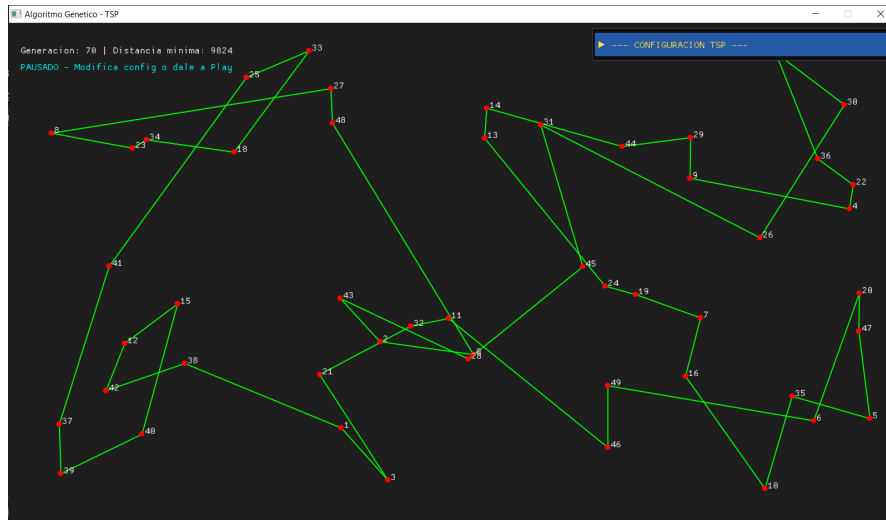


Figura 5: Generación 70: Formación de los primeros contornos claros del grafo del viajante (Distancia: 9824).

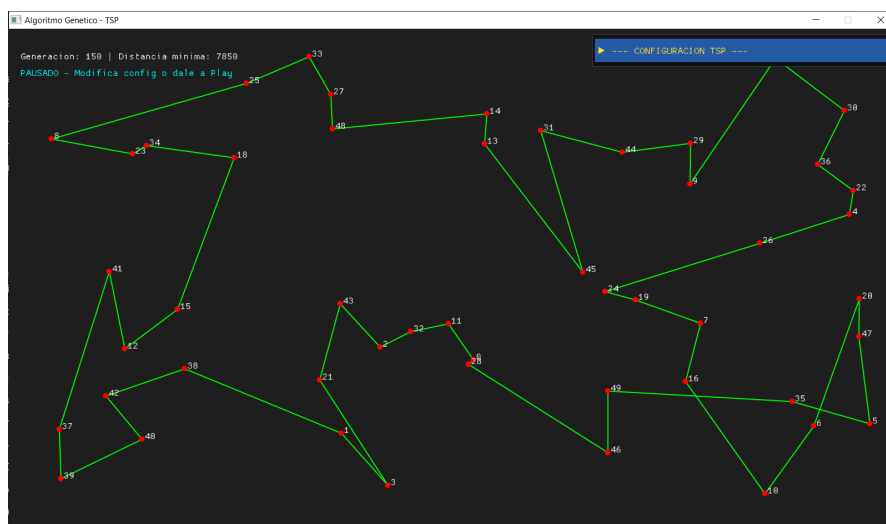


Figura 6: Generación 150: Afinamiento de rutas largas y eliminación de cuellos de botella (Distancia: 7850).

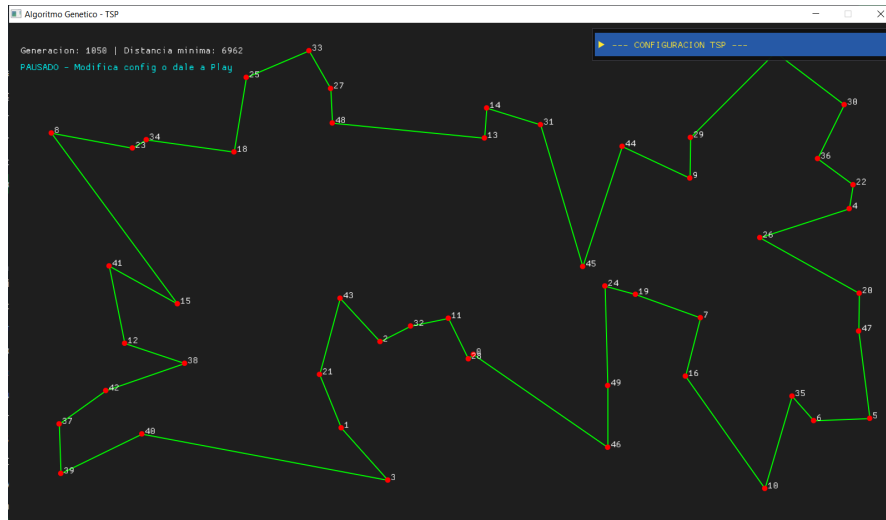


Figura 7: Generación 1050: Convergencia hacia una solución óptima o altamente subóptima sin auto-intersecciones visibles (Distancia: 6962).

4. Implementación del Algoritmo Genético

La lógica central del algoritmo se encuentra estructurada en `algoritmo_genetico.cpp`. Para representar el problema, se utilizan estructuras que encapsulan las posiciones en un plano 2D y el orden de visita (ruta) de cada individuo.

A continuación, se presenta un fragmento clave del código responsable de mutar a un individuo, demostrando la implementación de los operadores *Swap* (Intercambio) y *2-opt* (Inversión):

```

1 void mutate(Individual& ind, const GAConfig& config) {
2     uniform_real_distribution<double> dist_prob(0.0, 1.0);
3     // Evaluamos la probabilidad de mutar seg n la tasa
4     // configurada
5     if (dist_prob(rng) < config.mutation_rate) {
6         uniform_int_distribution<int> dist_city(0, config.
7             num_cities - 1);
8         int i = dist_city(rng);
9         int j = dist_city(rng);
10
11         if (config.mutation_mode == MutationMode::SWAP) {
12             // Intercambiamos la posici n de dos ciudades
13             swap(ind.route[i], ind.route[j]);
14         }
15         else if (config.mutation_mode == MutationMode::INVERSION)
16         {
17             // Invertimos un segmento completo de la ruta
18             if (i > j) swap(i, j);
19             reverse(ind.route.begin() + i, ind.route.begin() + j +
20                 1);
21         }
22     }
23 }

```


De igual manera, la recombinación genética o *Crossover* juega un rol crucial para heredar características buenas. El siguiente fragmento muestra cómo se manejan los métodos de cruce OX (*Order Crossover*) y PMX (*Partially Mapped Crossover*), cuidando siempre de no duplicar ciudades en la ruta generada:

```

1 Individual crossover(const Individual& p1, const Individual& p2,
2   const GAConfig& config) {
3   Individual child;
4   child.route.assign(config.num_cities, -1);
5
6   uniform_int_distribution<int> dist_city(0, config.num_cities -
7     1);
8   int start = dist_city(rng), end = dist_city(rng);
9   if (start > end) swap(start, end);
10
11  if (config.crossover_mode == CrossoverMode::OX) {
12    // OX: Se copia un segmento de p1 y se rellena el resto
13    // con p2 en orden
14    for (int i = start; i <= end; ++i) child.route[i] = p1.
15      route[i];
16
17    int childIdx = (end + 1) % config.num_cities;
18    for (int i = 0; i < config.num_cities; ++i) {
19      int cityP2 = p2.route[(end + 1 + i) % config.
20        num_cities];
21      if (find(child.route.begin(), child.route.end(),
22        cityP2) == child.route.end()) {
23        child.route[childIdx] = cityP2;
24        childIdx = (childIdx + 1) % config.num_cities;
25      }
26    }
27  }
28  else if (config.crossover_mode == CrossoverMode::PMX) {
29    // PMX: Preservacion de la posicion y resolucion de
30    // conflictos por mapeo
31    for (int i = start; i <= end; ++i) child.route[i] = p1.
32      route[i];
33
34    for (int i = start; i <= end; ++i) {
35      int cityP2 = p2.route[i];
36      if (find(child.route.begin(), child.route.end(),
37        cityP2) == child.route.end()) {
38        int current = p1.route[i];
39        int idxP2 = find(p2.route.begin(), p2.route.end(),
40          current) - p2.route.begin();
41        while (start <= idxP2 && idxP2 <= end) {
42          current = p1.route[idxP2];
43          idxP2 = find(p2.route.begin(), p2.route.end(),
44            current) - p2.route.begin();
45        }
46        child.route[idxP2] = cityP2;
47      }
48    }
49  }
50  for (int i = 0; i < config.num_cities; ++i) {
51    if (child.route[i] == -1) child.route[i] = p2.route[i];
52  }
53 }

```

```
40     }  
41 }  
42 return child;  
43 }
```

La flexibilidad de este diseño radica en que el estado de la estructura `GAConfig` es modificado en tiempo real por el panel de `ImGui` en el hilo principal, y sus valores dictan el comportamiento exacto de los operadores genéticos en el momento en el que se ejecuta cada ciclo o "generación" de la simulación.

5. Conclusiones

El programa desarrollado cumple con éxito el objetivo de demostrar visual y empíricamente el funcionamiento de un algoritmo genético frente a un problema NP-Hard como el TSP. La capacidad de pausar, modificar la mutación o cambiar el tipo de cruce en tiempo real, permite comprender la sensibilidad del algoritmo frente a sus hiperparámetros.

Anexos

El código fuente completo de este proyecto, incluyendo las librerías gráficas integradas y las instrucciones de compilación con `CMake`, se encuentra disponible de forma pública.

Repositorio en GitHub: <https://github.com/Aferrov/genetico>